

Normalização de Banco de Dados Relacional

Organização, Integridade e Eficiência no Armazenamento de Dados

Conteúdo:

Conceitos essenciais e aplicação prática das Formas Normais.

Nome do apresentador / Data.

O que é Normalização?

- **Definição:** Processo formal para organizar colunas e tabelas de um banco de dados.
- **Objetivo principal:** Minimizar a redundância de dados e evitar anomalias.
- **Garantia:** Assegurar a integridade referencial e consistência das informações.

Por que Normalizar?

- Sem a normalização, os bancos de dados sofrem com três grandes problemas:
- **Anomalia de Inserção:** Impossibilidade de adicionar dados por falta de outras informações.
- **Anomalia de Atualização:** Modificar um dado em um lugar e esquecer de atualizar suas cópias.
- **Anomalia de Exclusão:** Perder uma informação importante ao deletar um registro secundário.

Nosso Modelo de Exemplo

- **Conteúdo:**

- Para entender as regras, utilizaremos o seguinte esquema de banco de dados comercial:
- **CLIENTE e FORNECEDOR vinculados a uma CIDADE.**
- Processo de COMPRA (de fornecedores) e suas tabelas de itens (COMPRA_ITENS).
- Processo de VENDA (para clientes) e suas tabelas de itens (VENDA_ITENS).
- Cadastro centralizado de PRODUTO.

Primeira Forma Normal (1FN) - Teoria

1FN: Entidades e Atributos Atômicos

- **Conteúdo:**
 - **Regra principal:** Todos os atributos devem ser atômicos (indivisíveis).
 - **Proibições:** Não são permitidos atributos multivalorados (ex: dois telefones na mesma célula) nem atributos compostos.
 - **Exigência:** Cada tabela deve possuir uma Chave Primária (PK) definida.

Primeira Forma Normal (1FN) - Prática

Aplicando a 1FN no Exemplo

- **Conteúdo:**

- Imagine que na tabela `CLIENTE`, o campo `CLI_FONE` guardasse múltiplos números separados por vírgula ("9999-1111, 9999-2222").
- **Correção 1FN:** O campo deve conter apenas um valor por linha, ou os telefones devem ser extraídos para uma tabela filha.
- Tabela correta na 1FN:
 - `CLIENTE` (**`CLI_CODIGO`**, `CLI_NOME`, `CLI_ENDERECO`, `CLI_FONE`, `CLI_DT_NASC`, `CID_CODIGO`) -> Onde cada linha possui apenas um telefone individual.

Segunda Forma Normal (2FN) - Teoria

2FN: Dependência Funcional Total

- **Conteúdo:**

- **Pré-requisito:** A tabela já deve estar obrigatoriamente na 1FN.
- **Regra principal:** Todos os atributos não-chave devem depender da **chave primária total**, e não apenas de parte dela.
- **Foco:** Aplica-se principalmente a tabelas com chaves primárias compostas (compostas por duas ou mais colunas).

Segunda Forma Normal (2FN) - Prática

Aplicando a 2FN no Exemplo

- **Conteúdo:**

- Analise a tabela VENDA_ITENS (**PRD_CODIGO**, **VEN_CODIGO**, VL_UNIT, QT, VL_TOTAL).
- A chave primária é composta por PRD_CODIGO + VEN_CODIGO.
- O valor unitário (VL_UNIT) e a quantidade (QT) dependem do produto e daquela venda específica. Dependência total!
- Se houvesse um campo "PRD_DESCRICAO" dentro dessa tabela, ele dependeria apenas de PRD_CODIGO (parte da chave). Isso violaria a 2FN. Por isso, a descrição fica isolada na tabela PRODUTO.

Terceira Forma Normal (3FN) - Teoria

3FN: Eliminando Dependências Transitivas

- **Conteúdo:**

- **Pré-requisito:** A tabela deve estar estritamente na 2FN.
- **Regra principal:** Atributos não-chave não podem depender de outros atributos também não-chave.
- **Conceito:** A dependência deve ser direta da Chave Primária, sem intermediários (X determina Y, Y determina Z; logo, X determina Z indiretamente).

Terceira Forma Normal (3FN) - Prática

Aplicando a 3FN no Exemplo

- **Conteúdo:**

- Considere a relação de CLIENTE e CIDADE.
- Se na tabela CLIENTE colocássemos CID_NOME e CID_UF, teríamos um problema: o nome da cidade depende do código da cidade, que por sua vez depende do código do cliente.
- **Solução do exemplo:** Separação correta das tabelas:
 - CLIENTE (**CLI_CODIGO**, CLI_NOME, CLI_ENDERECO, CLI_FONE, CLI_DT_NASC, CID_CODIGO)
 - CIDADE (**CID_CODIGO**, CID_NOME, CID_UF)
- O cliente aponta apenas para a chave estrangeira CID_CODIGO. Dados da cidade ficam restritos à sua própria tabela.

Forma Normal de Boyce-Codd (BCNF)

Além da 3FN: A Forma Normal de Boyce-Codd

- **Conteúdo:**

- Uma versão ligeiramente mais forte e rigorosa da 3FN.
- **Regra:** Para toda dependência funcional não-trivial $X \twoheadrightarrow Y$, X deve ser obrigatoriamente uma chave candidata.
- No nosso modelo exemplo, as tabelas COMPRA_ITENS e VENDA_ITENS respeitam a BCNF porque suas dependências partem estritamente das chaves compostas identificadoras.

Quarta Forma Normal (4FN)

4FN: Dependências Multivaloradas Independentes

- **Conteúdo:**
 - **Conceito:** Trata de casos onde uma única tabela possui dois ou mais atributos multivalorados independentes entre si.
 - **Exemplo Teórico:** Se um FORNECEDOR pudesse ter várias certificações e atuar em vários ramos, misturar isso na mesma tabela geraria combinações redundantes desnecessárias.
 - **Solução:** Isolar as entidades em tabelas associativas separadas para que os múltiplos valores de uma não multipliquem os registros da outra.

Quinta Forma Normal (5FN)

5FN: Restrição de Junção (Join Dependency)

- **Conteúdo:**

- Conhecida também como Forma Normal de Projeção-Junção (PJNF).
- **Regra:** Uma tabela está na 5FN se ela não puder ser decomposta em tabelas menores sem que, ao juntá-las novamente (Join), ocorra perda de dados ou geração de dados fantasmas.
- Garante que relações ternárias (envolvendo 3 entidades simultaneamente) sejam reconstruídas com total fidelidade e precisão matemática.

Resumo do Modelo Normalizado

Visão Geral do Banco de Dados Proposto

- **Conteúdo:**

- O modelo enviado já se encontra perfeitamente normalizado até a 3FN/BCNF:
- **Tabelas de Cadastro:** CLIENTE, FORNECEDOR, CIDADE, PRODUTO (Sem redundâncias de nomes/endereços).
- **Tabelas de Movimento:** COMPRA, VENDA (Guardam o cabeçalho e os totais da operação).
- **Tabelas de Detalhe (N para N):** COMPRA_ITENS, VENDA_ITENS (Chaves compostas mapeando o relacionamento perfeito entre os produtos e os documentos de estoque).

Conclusão

Vantagens e Boas Práticas

- **Conteúdo:**
 - **Economia de espaço:** Dados armazenados uma única vez.
 - **Consistência:** Atualizar o nome de uma Cidade ou Produto altera instantaneamente todo o sistema.
 - **Desempenho:** Tabelas menores geram indexações mais rápidas e buscas mais eficientes.